Finio — Architecture & Engineering Guide

A complete tour of how Finio works: the data flow, every file's job, the AI/ML concepts behind it, the key design decisions, and an interview cheat-sheet so you can explain any part with confidence.

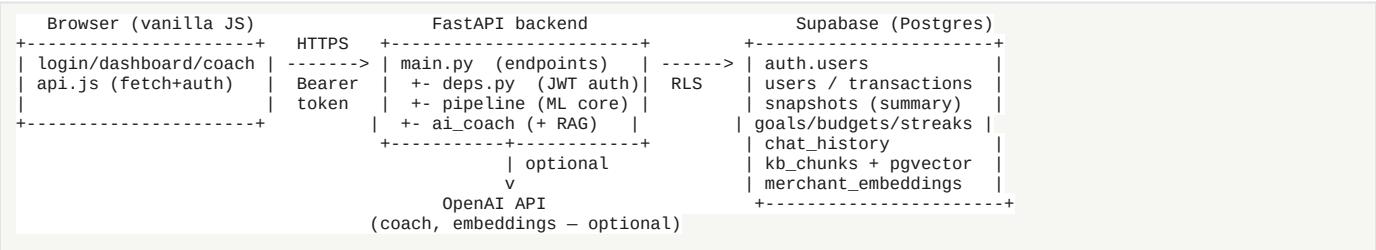
> Finio takes a messy bank statement and turns it into structured insight: categorised spending, budgets, bills, anomalies, forecasts, and an AI coach.

---

1. The one-paragraph explanation

> "Finio is a full-stack AI personal-finance app. A user uploads a bank statement; the backend parses it, classifies every transaction as income / spending / transfer, categorises spending with a rules-first + Naive-Bayes ML pipeline that learns from user corrections, then computes budgets, recurring bills, anomalies, and forecasts. On top of that sits an LLM coach that answers questions about the user's real numbers using tool-calling, grounded in a finance knowledge base via retrieval (RAG). It's FastAPI + scikit-learn + Supabase (Postgres/Auth/pgvector), with a dependency-free JS frontend, and it degrades gracefully to work fully offline without any API key."

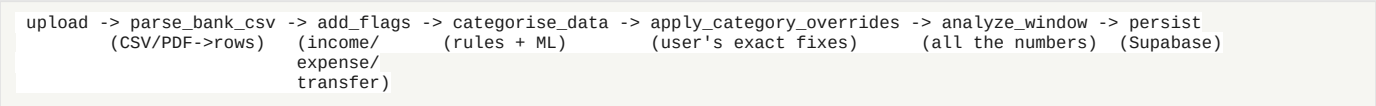
2. The big picture



Three ideas that explain most of the design:

1. Analyse once, re-slice by period. On upload the whole history is parsed, flagged and categorised, and the raw transactions are stored. Every later view (monthly / weekly / all-time) is recomputed by re-slicing those stored rows — no re-upload needed. (`pipeline.analyze_window` + `period.resolve_periods`.)
2. Snapshot pattern. The computed dashboard (metrics, budgets, forecasts…) is cached as one `summary_json` blob per user in the `snapshots` table, so a page load is a single read. User corrections (`overrides`, `custom_categories`) live inside that same JSON — no extra tables.
3. Graceful degradation. Every AI feature has a no-key fallback: LLM coach → rule-based coach; semantic RAG → TF-IDF; LLM insight → template. The app is fully usable with zero API keys.

3. The core data pipeline (the heart of the app)



- Parse (`bank_parser.py`, `pdf_parser.py`) → a normalised DataFrame: `date, amount, description, balance`.
- Flag (`data_processor.add_flags`) → adds a `flow` column: income, expense, or transfer. Transfers (own-account / P2P) are excluded from all totals so they never fake-inflate income or spend.
- Categorise (`categoriser.categorise_data`) → a spend category per expense.
- Override (`apply_category_overrides`) → the user's exact corrections win.

- Analyse (`pipeline.analyze_window`) → slices to the requested period and computes metrics, bills, anomalies, budgets, forecasts, personality, coach context — everything the dashboard shows.
- Persist (`db.persist_analysis`) → stores raw transactions + the summary JSON.

---

4. Backend file-by-file

Entry point & plumbing

File	What it does
`main.py`	FastAPI app + all HTTP endpoints (`/analyze`, `/dashboard`, `/transactions`, `/overrides`, `/goal`, `/profile`, `/spend-check`, `/coach`, `/insight`, `/what-if*`, `/reanalyze`). Also the in-process view cache (`_VIEW_CACHE`) that memoises re-sliced period views and clears on every write.
`api/deps.py`	Auth dependency. `get_current_user` parses the `Bearer` token and resolves the user id; `require_db` guards missing config.
`schemas.py`	Pydantic request models (validation) — `CoachRequest`, `GoalRequest`, `SpendCheckRequest`, `ProfileRequest`, `OverrideRule/Request`.
`config.py`	All constants: category rules & keywords, transfer keywords, model names (`OPENAI_MODEL`, `EMBED_MODEL`), thresholds, the disclaimer.

Parsing (messy input → clean rows)

File	What it does
`bank_parser.py`	Loads a CSV, auto-detects columns across bank formats (`_find_column`), handles header/headerless files, normalises to the standard schema, and cleans merchant names (`clean_merchant_name`) — stripping card numbers, locations, `SQ *`/`PAYID` noise.
`pdf_parser.py`	Extracts transactions from text-based PDF statements (pdfplumber), reconstructs amounts/signs from a running balance.

Core processing

File	What it does
`data_processor.py`	The classification core. `add_flags` sets `flow` (income/expense/transfer) and applies flow overrides. `compute_metrics` is the single source of truth for `total_income`, `total_spent`, `net_saved`, `savings_rate`, `latest_balance`, `daily_burn_rate`. `tx_key`/`key_series` give each transaction a stable id (occurrence-indexed so duplicates don't collide). `apply_flow_overrides` / `apply_category_overrides` apply user corrections by text-match or exact `tx_key`.

period.py	`resolve_periods` turns "monthly/weekly/daily/all/custom" into a concrete `[start, end]` window (anchored to the latest transaction), plus a prior window used as the budget baseline. `filter_window` slices the DataFrame.
categoriser.py	Hybrid categoriser (see §6). Rules first (`rule_category`), Naive-Bayes ML for the rest (`get_model`, cached), and active learning (`examples_from_overrides` augments the model with the user's corrections).

Analysis (rows → insight)

File	What it does
analytics.py	`category_breakdown` (spend by category %), `detect_patterns` (behavioural flags), `risk_score` + label.
bill_detector.py	Statistical recurring-bill detection: groups by merchant, checks amount stability (coefficient of variation) and interval regularity → monthly/fortnightly/weekly bills.
budget_setter.py	`suggest_budgets` — per-category limits from user targets → prior-period baseline → actual+headroom, with a zero-budget guard.
anomaly.py	`detect_anomalies` — per-category z-score over history flags unusually large charges (personalised outliers).
personality.py	Assigns a "money personality" + an action plan from spend patterns and savings rate.
savings_forecaster.py	`recommend_goal` (achievable target from monthly surplus), `forecast_goal` (on-track projection to a target date), `forecast_spending` (next-month projection + balance runway), `monthly_net_average` (robust monthly resample).
spend_check.py	`check_purchase` — green/yellow/red verdict on a planned purchase using the real account balance (falls back to period net savings).
invest.py	50/30/20 split, invest-readiness gate, ETF nudge, "first $1,000" plan, investment menu.
history.py	`build_snapshot` (the summary object), `update_streak` (upload streak; same-day no-inflate).

AI layer

File	What it does
ai_coach.py	The LLM coach. `build_context` compresses the user's finances into a JSON context. `coach_chat` runs a tool-calling loop (OpenAI function calling) over typed tools in `run_tool` (`get_income`, `category_total`, `filter_transactions`, `spend_check`, `lookup_concept`); `fallback_coach_response` answers without a key. `generate_insight` writes a monthly recap (LLM or template). Every answer carries the disclaimer.

rag.py	Retrieval. search tries semantic (embed query → pgvector match_kb) then falls back to TF-IDF over data/kb/*.md. Same output either way.
embeddings.py	embed_texts — batched OpenAI text-embedding-3-small; returns None with no key so callers fall back.

Orchestration & persistence

File	What it does
pipeline.py	Ties it all together. run_full_pipeline (fresh upload) and analyze_stored (re-slice stored history) both funnel into analyze_window, which computes every dashboard number for one period. _records serialises rows with tx_key; _restore_full_df rebuilds a full analysis frame from stored rows.
db.py	Supabase persistence. Transactions, snapshots (summary_json), goals, budgets, streaks, chat, profile. get_user_id verifies the JWT locally when SUPABASE_JWT_SECRET is set (else network). pgvector helpers: upsert_kb_chunks, match_kb, upsert_merchant_embeddings, match_merchants.

5. Frontend (dependency-free)

- **api.js** — the shared client: apiFetch (adds the Supabase Bearer token, handles 401/404/network errors), escapeHtml (XSS-safe rendering), money/date formatters, the platform-wide period selector (persisted in localStorage), and setupNav (top-right account menu).
- **dashboard.html** — the main page: metric cards, hero summary, savings goal, budgets, the interactive transaction editor, anomaly card, spend outlook, AI insight.
- **coach.html** / **patterns.html** / **invest.html** / **spend-check.html** / **profile.html** / **login.html** / **index.html** — the other pages.
- **styles.css** — a "receipt" theme with light/dark support.

6. AI / ML deep dives — what to know as an engineer

6.1 The hybrid categoriser (rules + Naive Bayes + active learning)

- Why rules first? Deterministic keyword rules (CATEGORY_RULES) are 100% predictable and need no data — e.g. "WOOLWORTHS" → Groceries. They handle the common cases exactly; the ML model only fills the gaps.
- The model: CountVectorizer (bag-of-words: turns a merchant string into word counts) → Multinomial Naive Bayes (a probabilistic classifier well-suited to word-count features; fast, tiny, interpretable). Trained on data/training_merchants.csv.
- Caching: the model trains once per process (get_model, module-level singleton) — previously it retrained on every request (a real perf bug we fixed).
- Active learning (the interesting part): when a user re-categorises a transaction, that correction (examples_from_overrides) is added to the training set (weighted ×3) and the model is retrained for that user and cached by a signature of their examples. On the next upload, new-but-similar merchants get the user's preferred category. This closes the loop: the product's UX generates labelled data that improves the model.
- Talking point: "It's a hybrid: deterministic rules for precision, ML for coverage, and a human-in-the-loop active-learning layer so it personalises."

6.2 RAG (retrieval-augmented generation)

- The problem it solves: an LLM hallucinates specifics. RAG retrieves real sourced text first, then lets the model answer grounded in it.
- Two implementations, one interface (`rag.search`):
- TF-IDF (default, no key): keyword-weighting + cosine similarity over the KB files. Matches on shared words.
- Semantic (with key + migration): embed the query with `text-embedding-3-small`, find nearest KB chunks in pgvector via cosine distance (`match_kb`). Matches on meaning — "saving for a rainy day" finds the emergency fund doc even with no shared words.
- Talking point: "I built retrieval twice — TF-IDF then embeddings on pgvector — so I can speak to the precision/latency/cost trade-off. It degrades gracefully: no key → TF-IDF, so retrieval always works."

6.3 The tool-using coach (LLM function-calling)

- The coach doesn't guess numbers. It's given typed tools (`run_tool`) and the model decides which to call; the backend executes them on the real transactions and feeds results back, up to 4 rounds (`MAX_TOOL_ROUNDS`). So "how much on coffee" returns an exact figure, not an estimate.
- Grounding + guardrails: the system prompt + disclaimer keep it from giving personalised buy/sell advice (a regulated domain). `lookup_concept` is a RAG tool.
- Talking point: "It's an agent loop with function-calling — the LLM plans, calls tools to compute on real data, and I control the tools and guardrails."

6.4 Anomaly detection (per-category z-score)

- For each category, compute mean & std of spend; a transaction whose z-score (`(x − mean) / std`) exceeds 2.5 is "unusual for you." Personalised, no model to train, explainable. (`anomaly.detect_anomalies`.)

6.5 Forecasting (robust monthly resample)

- Instead of fitting a line through noisy daily data, spend/savings are resampled by calendar month and averaged (`monthly_net_average`). One big payday or a lean month no longer skews the trend. Runway = `balance / daily_burn`.

6.6 Cross-cutting principle: graceful degradation

- `has_llm()` / `has_embeddings()` gate every AI call; each has a deterministic fallback. This makes the app demoable offline and cheap to run — a deliberate engineering choice worth calling out.

7. Data model (Supabase / Postgres)

Table	Holds	Notes
`users`	profile (email, age, income_bracket)	1:1 with `auth.use`
`transactions`	raw rows (date, amount, merchant, category, is_expense)	the source of truth; re-sliced per period
`snapshots`	`summary_json` — the computed dashboard	also stores `override` `custom_c` `anomalie` `spend_fo`

`goals` / `budgets` / `streaks`	savings goal, per-category limits, upload streak	
`chat_history`	coach messages	
`kb_chunks`	KB docs + embeddings (global)	pgvector; `match_kb`
`merchant_embeddings`	per-user merchant vectors	pgvector; RLS per user; `match_me`

Security: every user table has Row Level Security (`user_id = auth.uid()`), so the anon key in the browser can only ever read/write that user's rows. Secrets live in `.env` (gitignored); the anon key is public by design.

8. Key design decisions (and the trade-offs)

- Store raw + snapshot, re-slice on read → fast page loads, any period without re-upload; cost is recompute on period change (mitigated by the view cache).
- Corrections in **`summary_json`**, not new tables → simpler schema, corrections travel with the snapshot; cost is they're per-snapshot, not globally normalised.
- Rules-first categoriser → precision + no cold-start; cost is maintaining rules.
- Everything degrades without a key → demoable/cheap; cost is the "best" AI needs setup.
- **`tx_key`** = hash(date|merchant|amount|occ) → stable identity for single-row edits across re-slices; occurrence index avoids duplicate collisions.

9. Interview cheat-sheet (likely questions)

- "Walk me through what happens on upload." Parse → flag flow → categorise (rules+ML) → apply user overrides → analyse the period → store raw rows + a summary snapshot. (§3)
- "How does categorisation work / is it ML?" Hybrid — deterministic rules for the common merchants, Naive Bayes (bag-of-words) for the rest, plus active learning from user corrections. (§6.1)
- "What's RAG and where do you use it?" Retrieve grounded facts before generating. The coach's `lookup_concept` retrieves from a finance KB — TF-IDF by default, embeddings + pgvector when configured. (§6.2)
- "How does the coach avoid making up numbers?" Function-calling — it calls tools that compute on the real transactions; it never invents figures. (§6.3)
- "How is it secure?" Supabase Auth + Row Level Security per user, secrets in `.env`, XSS-escaped rendering, explicit CORS, optional local JWT verification. (§7)
- "How would you scale it?" Move the view cache to Redis, batch/queue embedding on upload, add a proper vector index (IVFFlat/HNSW) on pgvector, and a model registry
- eval harness for the categoriser.
- "What was the hardest bug?" e.g. `tx_key` collisions on duplicate transactions (two identical coffees edited together) — fixed with an occurrence index. (§8)

10. Glossary

- Bag-of-words / CountVectorizer — represent text as word counts.
- Naive Bayes — probabilistic classifier assuming feature independence; strong for word-count text classification.
- Embedding — a vector capturing a text's meaning; similar texts are near in space.
- Cosine similarity — angle-based closeness of two vectors (1 = identical).
- TF-IDF — weights words by how informative they are; keyword retrieval.
- pgvector — Postgres extension for storing/searching embeddings.

- RAG — retrieval-augmented generation (retrieve facts, then generate).
- Function calling / tool use — the LLM emits structured calls your code executes.
- RLS — Postgres Row Level Security; per-row access rules.
- z-score — how many standard deviations a value is from the mean.

11. Run & test

```
./run.sh                              # backend :8000 + frontend :5500
python -m tests.test_full_suite       # 69-case suite
python -m scripts.index_kb            # (optional) embed KB into pgvector
```

General information only, not financial advice.